

```

#include "queueLinkedList.h"

address newNode(ElType x, int pr) {
    Node *new = (Node *) malloc(sizeof(Node));
    if(new  $\neq$  NIL) {
        new→info = x;
        new→prio = pr;
        new→next = NIL;
    }
    return new;
}

void delNode(address P) {
    free(P);
}

bool isEmpty(PrioQueue q) {
    return (q.addrHead == NIL);
}

int length(PrioQueue q) {
    address p = q.addrHead;
    int count = 0;
    while(p  $\neq$  NIL) {
        count++;
        p = p→next;
    }
    return count;
}

void CreateQueue(PrioQueue *q) {

```

```

    q→addrHead = NIL;
}

void enqueue(PrioQueue *q, ElType x, int pr) {
    address new = newNode(x, pr);
    address curr, prev;
    if(new == NIL) {
        return;
    } else {
        if(isEmpty(*q) || new→prio < q→addrHead→prio) { // kosong
atau insert first saat 1 elemen
            new→next = q→addrHead;
            q→addrHead = new;
        }
        else {
            curr = q→addrHead;
            prev = NIL;
            while(curr ≠ NIL && curr→prio ≤ new→prio) {
                prev = curr;
                curr = curr→next;
            }
            new→next = curr;
            prev→next = new;
        }
    }
}

```

```

void dequeue(PrioQueue *q, ElType *x, int *pr) {
    address del = q→addrHead;
    *x = del→info;
    *pr = del→prio;
}

```

```
    q→addrHead = del→next;  
    delNode(del);  
}
```